

* Pointer to base class :-

- Pointer to base class point to objects as any derived class / base class.
- i.e. the pointer can access only the base class member
- function call via the pointer will also always execute the base class function, even if the derived class overrides the functions.

- following program shows pointer to base class concept:

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class Animal
```

```
{
```

```
public:
```

```
void speak ()
```

```
{
```

```
cout << "Animal sound " << endl;
```

```
}
```

```
};
```

```
class Dog : public Animal
```

```
{
```

```
public:
```

```
void speak ()
```

```
{
```

```
cout << "Dog barking sound " << endl;
```

```
}
```

```
};
```

```
void main ()
```

```
{
```

```
clrscr();
```

```
Animal *animalpte;
```

```

Dog obj ;
animalptε = &obj ;
animalptε → speak() ;
getch() ;
}

```

output —: Animal sound

* Pointer to Derived class :-

- A pointer to a derived class is a pointer that directly holds the address as derived class object.
- Derived class pointer can access both base class & derived class members.
- If a function is overridden, the derived class function is called instead of base class.
- A pointer to derived class cannot point to a base class objects.
- following example shows concept of pointer to derived class :

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class Base
```

```
{
```

```
public :
```

```
void basefun()
```

```
{
```

```
cout << "base class function:" << endl;
```

```
}
```

```
};
```

```
class Derived : public Base
{
```

```
    public:
```

```
        void derivedfun()
        {
```

```
            cout << "Derived class function:" << endl;
```

```
        }
```

```
};
```

```
void main()
```

```
{
```

```
    clrscr();
```

```
    Derived d;
```

```
    Derived *derivedptr = &d;
```

```
    derivedptr -> basefun();
```

```
    derivedptr -> derivedfun();
```

```
    getch();
```

```
}
```

output : Base class function

Derived class function

* Runtime polymorphism -

- Polymorphism is the key concept of oop's. It has following two types -

1) compile-time polymorphism

2) runtime polymorphism (virtual funⁿ)

- i.e. while doing programming the funⁿ's are get selected by the compiler at two ways ① At compile-time
② At runtime as program.

- runtime polymorphism is achieved by using virtual funⁿ. (i.e. function overriding).

* Virtual function -

- The function which are selected for execute / invoke compiler at runtime as program are called a virtual function.

- when base class function & derived class having function prototype at that time we declare the class function as virtual function by using 'virtual' keyword.

- when function is made virtual the compiler decide which funⁿ is used at runtime based on types, object pointed to the base class pointer.

• syntax to declare virtual funⁿ -

virtual return-type funⁿ-name (arg1, arg2, ..., argN)

Here,

virtual $\Rightarrow$ is keyword

return-type $\Rightarrow$ it is return-type as funⁿ.

funⁿ-name $\Rightarrow$ it is name of funⁿ.

* Characteristics / features / Rules of virtual function

1> virtual funⁿ must be memb. funⁿ of a class.

2> virtual funⁿ cannot be static.

3> virtual funⁿ can be accessed only by using object pointer only.

4> once you declare virtual funⁿ then we must to define it, even it is not used.

virtual funⁿ can be friend of another class.

If a virtual funⁿ is defined in base class then it necessary to redefine it in derived class.

W.A.P. to implement a concept of virtual function.

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class A
```

```
{
```

```
public :
```

```
virtual void show() ; // virtual funn
```

```
};
```

```
class B : public A
```

```
{
```

```
public :
```

```
void show() ;
```

```
};
```

```
void A::show()
```

```
{
```

```
cout << "show funn in class A : " << endl ;
```

```
}
```

```
void B::show()
```

```
{
```

```
cout << "show funn in class B : " << endl ;
```

```
}
```

```
void main()
```

```
{
```

```
A *obj ; // pointer of base class.
```

```
B obj1 ; // derived class object.
```

```
obj = &obj1 ; // base class pointer holds the  
address of derived class object.
```

```
obj -> show() ;
```

```
getch() ;
```

```
}
```

* pure virtual function -

- The virtual funⁿ equated / assigned to zero such as virtual funⁿ is called pure virtual funⁿ.
- pure virtual funⁿ are virtual funⁿ with no definition i.e. it has no implementation in base class. It is defined by derived class.
- In pure virtual funⁿ case the pointer compiler requires each derived class to either define the funⁿ or redefine it as pure virtual funⁿ.
- i.e. it is like empty bucket in base class & the derived class funⁿ have to fill it.

• syntax to declare pure virtual funⁿ -

virtual return-type funⁿ-name (arg1, arg2, ..., argN) = 0;

- eg. virtual void show () = 0;

• W.A.P. to implement pure virtual function:

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class pure
```

```
{
```

```
    public :
```

```
        virtual void show () = 0 ; // pure virtual fn.
```

```
};
```

```
class A : public pure
```

```
{
```

```
    public :
```

```

void show ()
{
    cout << " A class show fun" : "<< endl;
}

};

void main ()
{
    clrscr ();
    pure *p;
    A obj;
    p = &obj;
    p->show (); // call to pure virtual fn
    getch ();
}

```

* Abstract class -

- The class which contains pure virtual funⁿ are known as abstract class.
- The abstract class must contains at least one pure virtual funⁿ.
- The abstract class can provides the common defⁿ to all derived classes which are inherited from it.
- Let take one real life example
We have base class Animal & from that animal class we derive classes like human, dog & birds etc.
- Then every animal can sleep so, we can define the sleeping time in sleeping time in sleep() funⁿ in every class.

- So we declare sleep() funⁿ in animal (i.e. BaseClass) as pure virtual funⁿ & then implement that sleep funⁿ in every derived class.

* characteristics of Abstract class -

1> Abstract class is a must be base class (i.e. it is specially designed for as a base class).

2> The abstract class contains at least one pure virtual funⁿ.

3> The abstract class can't be instantiated (i.e. we can't create the instance / object as abstract class).

4> we can create pointer of an abstract class.

• W.A.P. to implement pure virtual funⁿ & abstract class in C++.

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class A
```

```
{
```

```
    public:
```

```
        virtual void show = 0;
```

```
};
```

```
class B : public A
```

```
{
```

```
    public:
```

```
        void show();
```

```
};
```

```
class C : public A
```

```

{
    public:
        void show();
}
void B::show()
{
    cout << "This is in derived class B:" << endl;
}
void C::show()
{
    cout << "This is in derived class C:" << endl;
}
void main()
{
    clrscr();
    A *p, *z;
    B obj1;
    C obj2;
    p = &obj1;
    z = &obj2;
    p -> show();
    z -> show();
    getch();
}

```

Virtual Destructor -

Virtual destructor is used to release / free / memory used by the child class (derived class) object when the child class is being removed from the memory by using parent class pointer object.

Virtual destructor is declared by using "virtual" keyword in base class.

• syntax to declare virtual destructor -

virtual ~class_Name ();

* Why we use virtual destructor -

- We know that the destructor is invoked automatically when the execution of a program is over

- i.e. object of class goes out of scope. But if we destroy object explicitly by using 'delete' operator through base class pointer to object then base class destructor is called.

- so result is it leaks the memory in our program.

- solution to make base class destructor virtual when we declare base class destructor as virtual then automatically it makes all derived class destructor as virtual even they do not have same name as the base class destructor.

- consider following example.

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class A
```

```
{
```

```
public:
```

```
    A ()
```

```
{
```

```
    cout << "This is default constructor in  
    base class:" << endl;
```

```
}
```

```

~A()
{
    cout << "This is destructor in base class:" << endl;
}

};

class B : public A
{
    public:
        B()
        {
            cout << "This is default constructor
            in derived class:" << endl;
        }
        ~B()
        {
            cout << "This is destructor in derived
            class:" << endl;
        }
}

```

};

void main()

{

clrscr();

A *obj;

B obj1;

obj = &obj1;

delete obj;

getch();

{

OIP- This is default constructor in Base class
 This is default constructor in Derived class
 This is destructor in base class.

- In above program we delete derived class object by using base class object pointer at that time only base class destructor is invoked not a derived class destructor is invoked.

- So solution to this we can make base destructor virtual.

• syntax for declaring virtual destructor -

```
virtual ~class-name()
{
    // body of destructor.
}
```

* container class -

- The class which contains object of another class as a its data member is called as container class

- The concept of declaring object of one class as data member into another class are known as composition.

- The container class only access public data of another class.

- It can not access private/protected data of another class.

- when we declare object of one class as data member into another class at that time has-a relationship occurred.

W.A.P. to implement a container class.

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class Base
```

```
{
```

```
public:
```

```
int a, b;
```

```
};
```

```
class B
```

```
{
```

```
Base ob;
```

```
public:
```

```
int c;
```

```
void get();
```

```
void add();
```

```
};
```

```
void B::get()
```

```
{
```

```
cout << "enter value of a=";
```

```
cin >> ob.a;
```

```
cout << "enter value of b=";
```

```
cin >> ob.b;
```

```
}
```

```
void B::add()
```

```
{
```

```
c = ob.a + ob.b;
```

```
cout << "addition =" << c;
```

```
}
```

```
void main()
```

```
{
```

```
clrscr();
```

```
B obj;
```

```
obj.get();
```

```
obj.add();
```

```
getch();
```

```
}
```

* member function overloading -

- function overloading means defining function with the same name but different parameters.

(e.g. void print(), void print(int x))

- In inheritance if a derived class defines a function with same name as the base class name functions the base class functions is hidden by derived class functions.

- let's take one following examples-

```
class Base
```

```
{
```

```
    public:
```

```
        void print()
```

```
        {
```

```
            cout << "base class print" << endl;
```

```
        }
```

```
};
```

```
class derived : public Base
```

```
{
```

```
    public:
```

```
        void print(int x)
```

```
        {
```

```
            cout << "base class print" << endl;
```

```
        }
```

```
};
```

- In above example derived class print() function hides base class print() function.

- To unhide base class function into derived class we redefine base class funⁿ in the derived class funⁿ.
- i.e. Base :: print() in derive class member function.
- following program shows member funⁿ overloading.

```
#include <iostream.h>
#include <conio.h>

class Base
{
    public:
        void print ()
        {
            cout << "base class print () : " << endl;
        }
};

class derived : public Base
{
    public:
        void print ()
        {
            Base :: print ()
            cout << "derived class print : " << endl;
        }
};

void main ()
{
    clrscr();
    derived ob ;
    ob.print();
    getch();
}
```